



**Hochschule Aalen**  
UNIVERSITY OF APPLIED SCIENCES

**FESTO**

# **Masterarbeit**

## **Fakultät Optik & Mechatronik**

# **Modellbasierte Funktionsentwicklung**

## **von**

### **Betreuer:**

Prof. Dr.-Ing. Jürgen Baur (Hochschule Aalen)  
Prof. Dr.-Ing. Fabian Holzwarth (Hochschule Aalen)  
Dr. Sergio Delgado (Festo SE & Co. KG)

### **Steffen Sieder**

Mechatronik / Systems Engineering Master  
88273  
3. Semester

Oberhausen 11  
73098 Rechberghausen  
+49 157 78321760  
siederst@online.de

**Zeitraum:** April 2026 - Oktober 2026

# Sperrvermerk

Die nachfolgende Masterarbeit beinhaltet vertrauliche und interne Daten der Festo SE & Co. KG Veröffentlichungen und Vervielfältigungen - auch nur auszugsweise - sind ohne ausdrückliche schriftliche Genehmigung des Unternehmens nicht gestattet.

Die Arbeit ist nur den Gutachtern sowie den Mitgliedern des Prüfungsausschusses zugänglich zu machen.

# Eidesstattliche Erklärung

Ich erkläre hiermit, dass ich die vorliegende Arbeit selbstständig und ohne Benutzung anderer als der angegebenen Hilfsmittel angefertigt habe. Die aus fremden Quellen direkt oder indirekt übernommenen Gedanken sind als solche kenntlich gemacht.

Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt und auch nicht veröffentlicht.

Esslingen den 27. April 2026  
(Ort, Datum, Unterschrift)

# Gender-Hinweis

In dieser Arbeit wird aus Gründen der besseren Lesbarkeit das generische Maskulinum verwendet. Weibliche und anderweitige Geschlechteridentitäten werden dabei ausdrücklich mitgemeint, soweit es für die Aussage erforderlich ist.

---

# Zusammenfassung

Die vorliegende Arbeit befasst sich mit der modellbasierten Funktionsentwicklung für Akkugeräte. Ziel ist es, zukünftig Funktionen für Akkugeräte modellbasiert in Matlab zu entwickeln und diese über die Autocodegenerierung in die Firmware zu integrieren. Im Rahmen dieser Arbeit werden erste Funktionen in die Simulationsumgebung übertragen und der Grundstein für die modellbasierte Funktionsentwicklung gelegt. Zunächst wird eine Modellstruktur sowie ein Ablauf für die Code-Generierung erarbeitet. Anschließend werden erste Maschinenschutzfunktionen in Matlab modelliert, in der Simulationsumgebung getestet und unter Berücksichtigung der Anforderungen aus dem Lastenheft verifiziert. In weiteren Systemtests werden an einer realen akkubetriebenen Motorsäge die Funktionen validiert.

Die abschließende Speicher- und Laufzeitanalyse zeigt, dass die modellbasierten Funktionen eine geringe Erhöhung des Ressourcenbedarfs im Vergleich zu den manuell codierten Funktionen verursachen. Dies ist vor allem auf die Schnittstelle zwischen dem Autocode und der Basissoftware zurückzuführen. Die Ergebnisse dieser Arbeit bieten eine solide Grundlage für den Übergang von der manuellen Programmierung hin zur modellbasierten Entwicklung, hinsichtlich nicht-sicherheitsrelevanter Funktionen.

---

# Inhaltsverzeichnis

|                                                                |            |
|----------------------------------------------------------------|------------|
| <b>Abbildungsverzeichnis</b>                                   | <b>III</b> |
| <b>Tabellenverzeichnis</b>                                     | <b>IV</b>  |
| <b>Skriptverzeichnis</b>                                       | <b>V</b>   |
| <b>1 Stand der Technik</b>                                     | <b>1</b>   |
| 1.1 Modellbasierte Systementwicklung . . . . .                 | 1          |
| 1.2 Simulink-Test . . . . .                                    | 2          |
| 1.3 Firmware Akkugeräte . . . . .                              | 3          |
| 1.4 Modellbasierte Entwicklungsumgebung - Playground . . . . . | 4          |
| 1.5 Modellierungsrichtlinien . . . . .                         | 6          |
| 1.5.1 Motivation . . . . .                                     | 6          |
| 1.5.2 Farbmatrix für Simulink-Blöcke . . . . .                 | 7          |
| 1.5.3 Namenskonvention . . . . .                               | 7          |
| <b>2 Fazit und Ausblick</b>                                    | <b>8</b>   |
| <b>A Programmcode</b>                                          | <b>10</b>  |
| <b>B Abkürzungen Matlab-Modell</b>                             | <b>14</b>  |

---

# Abbildungsverzeichnis

---

# Tabellenverzeichnis

|     |                                                 |    |
|-----|-------------------------------------------------|----|
| B.1 | Abkürzungsverzeichnis Matlab - Modell . . . . . | 14 |
|-----|-------------------------------------------------|----|

---

# Skriptverzeichnis

|     |                                                            |    |
|-----|------------------------------------------------------------|----|
| 1.1 | Ausschnitt aus einer Header-Datei einer Akku-Motorsäge [5] | 4  |
| 1.2 | Beispiel <code>#if</code> -Direktive                       | 4  |
| A.1 | Skript für die Autocodegenerierung und Nachbearbeitung     | 10 |

---

# 1 Stand der Technik

In diesem Kapitel wird der aktuelle Stand der Technik im Bereich der Entwicklung von Akkuprodukten aufgezeigt. Dafür erfolgt eine ausführliche Darstellung der aktuellen Firmware-Programmierung, der modellbasierten Entwicklungsumgebung sowie der geltenden Modellierungsrichtlinien, die im Rahmen der Entwicklung von Akkuprodukten bei STIHL angewendet werden.

## 1.1 Modellbasierte Systementwicklung

Der klassische Entwicklungsansatz stößt bei der Verwaltung und Verständlichkeit komplexer Systeme zunehmend an seine Grenzen. Änderungen in den frühen Entwicklungsphasen führen häufig zu Abstimmungsproblemen in den darauffolgenden Phasen und die Wiederverwendbarkeit der Ergebnisse zwischen diesen Phasen ist nur eingeschränkt möglich. Der schwerwiegendste Nachteil besteht jedoch darin, dass Tests zur Validierung und Verifikation erst am Ende der Entwicklungsphase durchgeführt werden können. Die modellbasierte Systementwicklung (engl. Model-Based Systems Engineering, MBSE) setzt hier an und zielt darauf ab, diese Nachteile zu beseitigen. Diese Art der Entwicklung basiert nicht nur auf Dokumenten, sondern vor allem auf Modellen [1].

Der modellbasierte Entwicklungsansatz zielt darauf ab, die Kommunikation zwischen Ingenieuren zu verbessern, indem eine einheitliche Grundlage für ausführbare Spezifikationen geschaffen werden. Zudem fördert die Verwendung von Modellen die Wiederverwendbarkeit einzelner Funktionen und Komponenten. Ein weiterer Vorteil des modellbasierten Ansatzes liegt in der variablen Betrachtungsweise (Anforderungssicht, funktionale Sicht, logische Sicht und physikalische Sicht) auf die Entwicklung, was die Verständlichkeit für alle Stakeholder (Interessengruppen) wesentlich erleichtert. Besonders hervorzuheben sind die frühzeitige Durchführung von Funktionstests zur Verifikation und der Einsatz von Autocodegeneratoren, welche die Implementierungszeit erheblich verkürzen [1].

Für einen zielführenden Einsatz des modellbasierten Ansatzes ist es erforderlich, alle Anforderungen der Stakeholder in Architekturmodellen zu integrieren. Dies ermöglicht die Erstellung einer intuitiven Systembeschreibung. Mithilfe der Matlab-Toolkette können die entwickelten Architekturmodelle nahtlos in Implementierungsmodelle überführt werden. Darüber hinaus unterstützt diese Toolkette die vollständige Durchführung des modellbasierten Entwicklungsprozesses innerhalb des V-Modells [1]. Das zugehörige V-Modell ist in ?? dargestellt.



In Matlab stehen für die verschiedenen Entwicklungsphasen spezifische Werkzeuge zur Verfügung. Systemanforderungen können in Simulink-Requirements definiert oder importiert und mithilfe des System-Composers mit den jeweiligen Funktionen verknüpft werden. Die Funktionen und physikalischen Eigenschaften des Systems lassen sich durch Tools wie Simulink, Simscape und Stateflow auf Subsystem- und Komponentenebene modellieren. In der Implementierungsphase kommen plattformspezifische Autocodegeneratoren, beispielsweise der Embedded-Coder, zum Einsatz. Für die Verifikation und Validierung der Systemfunktionen stehen verschiedene Methoden zur Verfügung. In einer frühen Entwicklungsphase findet die sogenannte Model-in-the-Loop-Simulation (MIL) Einsatz, bei der die entwickelten Modelle innerhalb der Simulationsumgebung getestet werden. Dadurch können Fehler bereits in einer frühen Entwicklungsphase identifiziert und behoben werden. Der Entwicklungsprozess im V-Modell kann darüber hinaus durch den Einsatz von Rapid-Control-Prototyping (RCP)-Systemen ergänzt werden [1].

### 1.2 Simulink-Test

In dieser Arbeit wird die Simulink-Test-Toolbox für das Testen einzelner Funktionen eingesetzt. Diese Toolbox stellt Werkzeuge zur Verfügung, um systematische und simulationsbasierte Tests von Simulink-Modellen zu erstellen, zu verwalten und durchzuführen. Darüber hinaus ermöglicht sie die Prüfung von generiertem Autocode sowie von simulierten und physischen Hardware-Komponenten.

Mit der Toolbox können sogenannte Test-Harnesses (Testumgebungen) erstellt werden, die es ermöglichen, die zu testende Komponente isoliert aus einem bestehenden Simulink-Modell zu analysieren [2]. Eine Übersicht über die Erstellung solcher Test-Harnesses ist in dargestellt.

Das System Under Test wird aus einem Gesamtmodell herausgelöst und in einer separaten Testumgebung isoliert. Innerhalb dieser Umgebung stehen vielfältige Möglichkeiten zur Verfügung, um die Komponenten umfassend zu testen. Für die Generierung von Testsignalen sowie die Bewertung der Ausgangssignale können verschiedene Blöcke genutzt werden. Der Testsequenz-Block ermöglicht beispielsweise die Konstruktion komplexer Testsequenzen und Bewertungslogiken.

Ein Test-Manager dient der Verwaltung und Steuerung sämtlicher Tests, sowohl für unterschiedliche Test-Harnesses als auch für mehrere Tests an einer einzelnen Komponente. Der Test-Manager bietet Vorlagen für Simulations-, Baseline- und Äquivalenztests, die eine Durchführung von Funktionstests, Unit-Tests, Regressions- und Back-to-Back-Tests

ermöglichen. Diese Tests können in verschiedenen Modi ausgeführt werden, darunter Model-in-the-Loop (MIL), Software-in-the-Loop (SIL), Processor-in-the-Loop (PIL) und Echtzeit-Hardware-in-the-Loop (HIL) [2].

Bei Simulationstests wird die zu testende Komponente simuliert und ihre Ausgänge werden mithilfe von Test-Assessment- oder Temporal-Assessment-Blöcken überprüft. Für den Vergleich der simulierten Komponente mit realen Messdaten stehen Baselinetests zur Verfügung. Diese ermöglichen den Abgleich der Simulationsergebnisse mit Messwerten aus der Praxis und die Verifikation anhand definierter Toleranzen. Äquivalenztests bieten die Möglichkeit, zwei unterschiedliche Tests auf Übereinstimmung zu vergleichen. Dies ist insbesondere für die Verifikation von generiertem Autocode von Bedeutung, da die Tests sowohl im Model-in-the-Loop (MIL)- als auch im Software-in-the-Loop (SIL)-Modus ausgeführt und miteinander abgeglichen werden können.

Durch die zusätzliche Verwendung der Requirements-Toolbox können Anforderungen aus dem Lastenheft direkt in Simulink hinterlegt und mit den entsprechenden Tests verknüpft werden. Ergänzend liefert Simulink Coverage Informationen zur Testabdeckung, wodurch die Vollständigkeit der durchgeführten Tests analysiert werden kann [2].

### 1.3 Firmware Akkugeräte

In diesem Abschnitt wird der Aufbau der Firmware für Akkugeräte näher erläutert. Die Firmware ist in der Programmiersprache C implementiert und wird bisher komplett händisch programmiert. Das gesamte Projekt setzt sich aus mehreren funktional voneinander getrennten C-Dateien zusammen. Jede C-Datei verfügt über eine eigene Header-Datei, in der die öffentlichen Funktionen und Konstanten definiert sind. Innerhalb einer C-Datei kommen lokale Funktionen zum Einsatz, welche auch als private Funktionen bezeichnet werden. Jede Datei beinhaltet mindestens eine öffentliche Funktion, die von anderen Dateien aufgerufen werden kann. Diese Funktionen fungieren als Schnittstelle zwischen den verschiedenen Funktionalitäten. Um eine Funktion aus einer anderen Datei nutzen zu können, ist es erforderlich, die Header-Datei der entsprechenden C-Datei einzubinden. Die meisten Funktionen verfügen über eine öffentliche Funktion, die zyklisch aufgerufen wird und wiederum die privaten Funktionen innerhalb einer Datei aufruft. In der Firmware werden dabei drei verschiedene Zykluszeiten genutzt (62.5 µs, 1 ms und 10 ms) [3].

Die Firmware ist so konzipiert, dass sie für eine Vielzahl unterschiedlicher Geräte, wie beispielsweise akkubetriebenen Motorsägen und Trennschleifer, verwendet werden kann. Für jedes Gerät existieren zudem eigene Baureihen mit spezifischen Funktionen. In der Firmware sind sämtliche Funktionen für alle Geräte integriert. Die einzelnen Funktionen

werden über eine maschinenabhängige Header-Datei mithilfe von `#if`-Direktiven gesteuert. Während des Kompilierungsprozesses werden diese Direktiven ausgewertet. Nur wenn die Anweisung zu einem logischen Wert von eins führt, wird der nachfolgende Code bis zur abschließenden `#endif`-Direktive in den Kompilierungsprozess einbezogen. Auf diese Weise werden ausschließlich die relevanten Funktionen für das ausgewählte Gerät eingebunden und der Speicherplatz des Steuergerätes nicht mit irrelevantem Code belastet [3] [4]. In Skript 1.1 ist ein Auszug aus der Header-Datei einer akkubetriebenen Motorsäge dargestellt.

```
1 /** @brief Used for modelbased development */
2 #define MODELBASED_PLAYGROUND          ENABLE_OPTION
3 /** @brief Activation of blockade detection */
4 #define ACTIVATE_BLOCKADE_MONITORING    ENABLE_OPTION
5 /** @brief Configure Motortype */
6 #define USED_MOTOR_CONTROL              EC_MOTOR_CONTROL
```

Skript 1.1: Ausschnitt aus einer Header-Datei einer Akku-Motorsäge [5]

In dieser Datei werden die Funktionen festgelegt, die während des Kompilierungsprozesses einbezogen werden. In Zeile zwei ist die Option für den Playground (eine Erläuterung hierzu erfolgt im nächsten Abschnitt) aktiviert, wodurch der aus Matlab generierte Autocode in den Kompilierungsprozess integriert wird. Damit dies funktioniert, muss die `#if`-Direktive an der entsprechenden Stelle im Code korrekt platziert werden (siehe Skript 1.2).

```
1 #if (MODELBASED_PLAYGROUND == ENABLE_OPTION)
2 // Funktions Code
3 #endif
```

Skript 1.2: Beispiel `#if`-Direktive

Sobald die Option `MODELBASED_PLAYGROUND` aktiviert ist, wird der zugehörige Funktionscode beim Kompilieren berücksichtigt.

## 1.4 Modellbasierte Entwicklungsumgebung - Playground

Für die automatische Codegenerierung aus Matlab existiert bereits ein Modell, welches nur wenige Funktionen umfasst, die in die Firmware integriert werden. Dieses Modell wird als

Playground bezeichnet. In diesem Modell sind sämtliche erforderlichen Einstellungen für die Codegenerierung bereits vorgenommen worden. Zudem ist die Schnittstelle zwischen dem Basiscode und dem automatisch generierten Code definiert und exemplarisch für die wenigen implementierten Funktionen validiert. Somit stellt dieses Modell die Grundlage für die nachfolgende Arbeit dar. Da das Modell jedoch mit einer älteren Matlab-Version erstellt wurde, sind zusätzliche Einstellungen für die Codegenerierung hinzugekommen bzw. bestehende Einstellungen wurden angepasst.

Nachfolgend wird die Schnittstelle zwischen der Basissoftware und dem automatisch generierten Code detailliert erläutert. Alle Parameter sowie die Eingangs- und Ausgangssignale werden in einem Simulink Data Dictionary definiert. Ein Ausschnitt davon ist in zu sehen.

Im Dictionary werden alle Parameter, Eingangs- und Ausgangssignale des Playgrounds sowie alle Signale definiert, die zu einem späteren Zeitpunkt bei Tests an der realen Hardware verfolgt werden sollen. Dieses Dictionary wird dem Playground zugeordnet, sodass die Variablen wie im normalen Matlab-Workspace verwendet werden können. Um die Eingangs- und Ausgangssignale als Struktur an den Playground zu übermitteln, werden sie im Dictionary als Bussignale angelegt. Dies erleichtert die Übergabe der einzelnen Variablen erheblich. Jeder Variable im Dictionary wird ein spezifischer Datentyp zugewiesen. Zusätzlich können eine Beschreibung und eine Einheit ergänzt werden, was die Lesbarkeit und Nachvollziehbarkeit der Variablen erheblich verbessert. Das Dictionary bildet die Grundlage für die spätere Autocodegenerierung. Das dazugehörige Matlab Modell ist in dargestellt.

Auf der linken Seite werden die einzelnen Eingangsgrößen aus dem Bussignal extrahiert und separat an den Playground übergeben. Auf der rechten Seite hingegen werden die Ausgangssignale erneut zu einem Bussignal zusammengeführt. Verschiedene Schutzfunktionen wirken sich auf die Motorsteuerungslimits aus, wie beispielsweise den maximal zulässigen Strom, die maximale Leistung und das maximale Drehmoment. Die Motorsteuerungslimits sind in der Basissoftware auf normierte 16bit Integer-Werte skaliert. Damit in den Berechnungen die tatsächlichen Grenzwerte für Strom, Leistung usw. verwendet werden können, müssen diese zunächst in float-Werte umgerechnet werden. Diese Umrechnung erfolgt in der Funktion `Q15_Float`. Um die Limits anschließend korrekt an die Basissoftware zurückzugeben, erfolgt die Rückumrechnung der Signale mithilfe der Funktion `Float_Q15`.

In der Basissoftware existiert eine C-Datei namens *Playground.c*, welche als Schnittstelle zwischen dem manuellen Code und dem automatisch generierten Code dient. Diese Datei umfasst alle Funktionen, die für die Bereitstellung der Eingangssignale sowie für die Verarbeitung der Ausgangssignale des Playgrounds erforderlich sind. Die Funktion *Playground\_run\_1ms* ist für das Einlesen der Eingangssignale zuständig und wird aus der

Basissoftware zyklisch aufgerufen. Im Rahmen der Autocodegenerierung wird eine Funktion mit der Bezeichnung *Modelbased\_playground\_run\_1ms* erstellt. Diese Funktion wird zyklisch aus der Basissoftware aufgerufen und übernimmt die Steuerung der im Autocode implementierten Funktionen.

Um die Funktion, welche die Eingangssignale bereitstellt, im Autocode aufzurufen, kann mittels spezieller Simulink-Blöcke an bestimmten Stellen des generierten Codes manueller C-Code eingefügt werden [6]. Eine beispielhafte Darstellung dieses Prozesses ist in. Im Abschnitt „System Outputs Function Declaration Code“ kann manueller C-Code am Beginn der zyklisch aufgerufenen Hauptfunktion *Modelbased\_playground\_run\_1ms* des Autocodes eingefügt werden. An dieser Stelle wird die Funktion `Playground_get_input()` aufgerufen, um die Eingangssignale einzulesen. Am Ende der Hauptfunktion besteht ebenfalls die Möglichkeit, manuellen Code über den Abschnitt „System Outputs Function Exit Code“ hinzuzufügen. Darüber wird die Struktur `MBSE_output`, welche alle Ausgangssignale des Playgrounds enthält, von der Funktion `Playground_set_output()` an die Basissoftware übergeben. Darüber hinaus stellt die Datei *Playground.c* eine öffentliche Funktion bereit, die es ermöglicht, die Ausgänge des Playgrounds aufzurufen und in anderen Funktionen in der Basissoftware weiterzuverarbeiten.

## 1.5 Modellierungsrichtlinien

Dieser Abschnitt befasst sich mit den unternehmensinternen Modellierungsrichtlinien von STIHL für Matlab-, Simulink- und Stateflow-Modelle sowie den festgelegten Namenskonventionen zur Benennung von Parametern und Signalen. Diese Richtlinien dienen als Grundlage für die modellbasierte Entwicklung und sind sowohl für externe Zulieferer als auch für die interne Entwicklung vorgeschrieben.

### 1.5.1 Motivation

Die Entwicklung neuer und zunehmend komplexerer Funktionen führt zu einer deutlichen Erweiterung des Funktionsumfangs. Viele dieser Modelle werden anschließend für die automatische Codegenerierung genutzt, wobei dabei bestimmte zentrale Aspekte berücksichtigt werden müssen. Die in [7] aufgeführten Richtlinien und Empfehlungen beziehen sich auf die automatische Codegenerierung aus Matlab-Modellen und dienen primär als Nachschlagewerk für die Funktionsentwicklung. Sie stellen zudem sicher, dass die Struktur der verschiedenen Modelle weitgehend einheitlich bleibt, was die Lesbarkeit und

Verständlichkeit der Modelle innerhalb des Unternehmens verbessert.

### 1.5.2 Farbmatrix für Simulink-Blöcke

Zur Verbesserung der Übersichtlichkeit der Simulink-Modelle existieren Vorgaben für die farbliche Gestaltung bestimmter Blöcke. In ist eine Tabelle mit den jeweiligen Blöcken und ihren zugewiesenen Farben dargestellt.

Durch die farbliche Kennzeichnung der Blöcke wird der jeweilige Blocktyp auf den ersten Blick erkennbar, ohne dass eine detaillierte Prüfung erforderlich ist. Dies trägt wesentlich zur Erhöhung der Lesbarkeit und Verständlichkeit des Modells bei.

### 1.5.3 Namenskonvention

Die Benennung von Funktionen und Signalen spielt in komplexen und umfangreichen Modellen eine entscheidende Rolle, weshalb STIHL ein eigenes Dokument zur Namenskonvention veröffentlicht hat [8]. Dieses Dokument definiert die Richtlinien zur Benennung von Funktionen, Parametern, Variablen und weiteren Elementen. Ziel dieser Konventionen ist es, eine verbesserte Lesbarkeit und Nachvollziehbarkeit der Modelle zu gewährleisten und somit die Qualität der Modelle zu erhöhen.

Die wichtigsten Regeln für die Namensgebung werden im Folgenden zusammengefasst. Die Signalnamen folgen einem einheitlichen Format: *qq(X)xxxYyyyZzzzOooo\_Pppp*. Parameter werden nach einem ähnlichen Schema benannt, jedoch wird dem Namen ein Präfix *P\_* vorangestellt. In dem verwendeten Playground sowie in der Basissoftware wurde bisher das Präfix *CPS* vor die Parameterbezeichnung gesetzt, weshalb diese Konvention in der vorliegenden Arbeit beibehalten wird.

Die jeweiligen Teilabkürzungen werden gemäß eines festgelegten Schemas gebildet, wobei sie einer Abkürzungsliste entnommen werden. In sind die einzelnen Elemente der Namenskonvention detailliert beschrieben.

Alle Benennungen innerhalb des Modells sollen diesem Muster folgen. Sollte jedoch ein Signal oder Parameter nicht ausreichend beschrieben sein, ist es gestattet, ein zusätzliches Element mit einer maximalen Länge von vier Zeichen hinzuzufügen. Die bestehende Abkürzungsliste wird in dieser Arbeit zur Benennung herangezogen und ergänzt. In Tabelle B.1 sind alle innerhalb des Matlab-Modells verwendeten Abkürzungen aufgeführt. Die letzte Spalte zeigt an, ob die jeweilige Abkürzung bereits in [8] definiert ist.

---

## 2 Fazit und Ausblick

Im Rahmen dieser Masterarbeit wurde die Grundlage für die modellbasierte Funktionentwicklung im Bereich der Akkuprodukte bei STIHL geschaffen. Durch eine sorgfältige Vorbereitung des Matlab-Modells, insbesondere hinsichtlich der Modellstruktur und der automatisierten Code-Generierung, konnte ein solides Fundament für die Arbeit gelegt werden. Mithilfe von Variant-Subsystemen wurde die bestehende modulare Firmware-Architektur erhalten und erweitert.

Eine umfassende Analyse des Lastenhefts sowie des bestehenden C-Codes bildete die Basis für die Modellierung und Implementierung der Maschinenschutzfunktionen. Mithilfe der Simulink-Test-Toolbox wurden die einzelnen Funktionen in einer isolierten Testumgebung unter Verwendung selbst definierter Testabläufe automatisiert geprüft. Durch die Verknüpfung der Anforderungen aus dem Lastenheft mit den definierten Testabläufen konnten die implementierten Funktionen automatisch verifiziert werden.

Der C-Code für die Maschinenschutzfunktionen wurden mittels automatisierter Codegenerierung durch den Embedded-Coder erstellt und in die bestehende Firmware integriert. Über eine Konfigurationsoption in der Header-Datei der jeweiligen Maschine ist es möglich, während der Erstellung des Maschinencodes zwischen den automatisch generierten Funktionen und den vorhandenen Funktionen der Basissoftware zu wechseln. Bei aktivierter Autocode-Option werden die entsprechenden Basissoftware-Funktionen deaktiviert und während des Kompilierens des gesamten Codes nicht berücksichtigt.

Im Rahmen nachfolgender Systemtests wurden die modellbasiert entwickelten Maschinenschutzfunktionen erfolgreich an einer akkubetriebenen Motorsäge geprüft und verifiziert. Eine abschließende Analyse des Speicherbedarfs und der Laufzeit bei aktiviertem und deaktiviertem Autocode veranschaulichte die Unterschiede zwischen dem Autocode und den manuell codierten Funktionen. Bei aktivierten Autocode-Funktionen erhöht sich der Speicherbedarf, primär bedingt durch die Schnittstelle zwischen der Basissoftware und dem Autocode, die für die Bereitstellung der Ein- und Ausgangssignale verantwortlich ist. Die Speichererhöhung bezogen auf den ROM beläuft sich lediglich auf 0,37 %, wenn der zusätzliche Speicherbedarf der Schnittstelle berücksichtigt wird, und auf 0,107 %, wenn ausschließlich die Funktionalität in Relation zum gesamten verfügbaren Speicher des Steuergeräts betrachtet wird. Auch die Laufzeit erhöht sich geringfügig aufgrund der zusätzlichen Abarbeitungszeit der Schnittstelle, was jedoch keinen signifikanten Einfluss auf die Gesamtperformance hat. Mit der zunehmenden Auslagerung von Funktionen aus der Basissoftware in den Playground relativiert sich der zusätzliche Speicherbedarf sowie die verlängerte Laufzeit der Schnittstelle.

Der modellbasierte Entwicklungsansatz führt im Vergleich zur manuellen Programmierung

zu einer geringfügigen Erhöhung des Speicherbedarfs und der Laufzeit. Jedoch überwiegen die Vorteile dieses Ansatzes. Die strukturierte Implementierung der Funktionen auf Modellebene verbessert deren Nachvollziehbarkeit und Verständlichkeit erheblich. Dadurch können präzise Anpassungen einzelner Funktionen effizient und mit geringem Aufwand realisiert werden.

Zur abschließenden Prüfung und Validierung der in dieser Arbeit entwickelten Maschinenschutzfunktionen auf der realen Hardware sollte die Maschine mit aktiviertem Playground den standardisierten Prüfablauf der Testing-Gruppe durchlaufen.

Die Fortführung des Themas der modellbasierten Funktionsentwicklung ermöglicht es, weitere Funktionen aus der bestehenden Basissoftware in das Matlab-Modell zu übertragen. Hierbei kann die erarbeitete Modellstruktur sowie die konfigurierte Autocodegenerierung verwendet werden. Darüber hinaus besteht die Möglichkeit, die bestehende Schnittstelle zwischen der Basissoftware und dem Autocode weiter zu verwenden, Anpassungen vorzunehmen oder bei Bedarf zu erweitern.

Zusätzliche Untersuchungen könnten sich auf die Speicherbelegung in Bezug auf die Autocode-Einstellungen und die Modellierung konzentrieren. Derzeit erfolgt die Signalübergabe zwischen den Maschinenschutzfunktionen im Autocode durch den Zugriff auf globale Variablen. Zur Optimierung des Speicherbedarfs sollte dieses Verfahren überprüft und gegebenenfalls angepasst werden, um den Speicherverbrauch weiter zu minimieren. Dadurch bestehen weiterhin Möglichkeiten, die Leistungsfähigkeit und Effizienz des Autocodes zu verbessern.

Parallel zu dieser Arbeit wird ein physikalisches Modell in Matlab entwickelt, welches das Gesamtverhalten von Akkugeräten simuliert. Dieses Modell soll sowohl die physikalischen Eigenschaften als auch die Softwarefunktionen enthalten. Die im Rahmen dieser Arbeit implementierten Maschinenschutzfunktionen, die in einem Referenced-Subsystem abgespeichert sind, lassen sich mühelos in das physikalische Modell integrieren. Dabei ist lediglich die Übergabe der Eingangssignale sowie die Verarbeitung der Ausgangssignale zu berücksichtigen.

Zusammengefasst wird deutlich, dass der modellbasierte Ansatz auch künftig vielfältige Einsatzmöglichkeiten bei STIHL im Bereich der Akkugeräte bietet und zusätzliche Vorteile bei der Produktentwicklung generiert.



---

# A Programmcode

```
1 %% Wichtig zu wissen!
2 % Ab Matlab 2023 kann die Datentyp Benennung für die Autocode Generierung
3 % angepasst werden, damit diese direkt mit der Firmware übereinstimmt.
4 % https://de.mathworks.com/help/ecoder/ref/datatypereplacement.html
5
6 %%
7 % Clear command window
8 clc
9 % close all files
10 fclose('all');
11 % Clear work folder and remove old build artefacts first
12 if exist('slprj','dir')
13     rmdir ('slprj', 's');
14 end
15 if exist([bdroot '_ert_rtw'], 'dir')
16     rmdir ([bdroot '_ert_rtw'], 's');
17 end
18 if exist(bdroot,'dir')
19     rmdir (bdroot, 's');
20 end
21 % create output directory
22 mkdir (bdroot);
23 delete *.slxc
24
25 %% build section
26 fprintf('\n\n');
27 fprintf('-----\n');
28 fprintf(' Code-Generation --> CPS_Data-structure \n');
29 fprintf('-----\n');
30 % Disable Verbose build mode
31 set_param(bdroot,'RTWVerbose',false);
32 % Build the model
33 rtwbuild(bdroot, 'generateCodeOnly', true);
34 % Enable Verbose build mode
35 set_param(bdroot,'RTWVerbose',true);
36
37 %% PostProcessing: generate CPS-fragment and rework source files
```

```
38 fprintf('\n\n');
39 fprintf('-----\n');
40 fprintf(' Code-Generation --> CPS_Fragment / Postprocessing \n');
41 fprintf('-----\n');
42 run create_A2L_fragment.m
43 copyfile('A2L_Fragment.c',bdroot);
44 delete 'A2L_Fragment.c'
45 run create_CPS_fragment.m
46 copyfile('CPS_Fragment.c',bdroot);
47 delete 'CPS_Fragment.c'
48 % Call ASAP2Creator to create A2L-Template based on ASAP2_Fragment
49 copyfile ../../000_Tools/002_A2L/ASAP2_Creator/ASAP2Creator.ini
50 copyfile ../../000_Tools/002_A2L/ASAP2_Creator/Master.a2l
51 copyfile ../../000_Tools/002_A2L/ASAP2_Creator/ASAP2Merger.ini
52 copyfile ../../000_Tools/002_A2L/ASAP2_Creator/Interface.a2l
53 system('ASAP2Creator');
54 system('ASAP2Merger -PASAP2Merger.ini -SModelbased_playground_Template.a2l
      -MInterface.a2l -OModelbased_playground_Template.a2l');
55 delete ASAP2Creator.ini Master.a2l ASAP2Merger.ini Interface.a2l
56 % Definitions
57 % define build directory
58 build_dir = [bdroot '_ert_rtw\'];
59 % define ignored files
60 % ignore_files_h = [strcat(bdroot,"_types.h"), strcat(bdroot,"_private.h"),
      "Product_parameter.h","rtmodel.h", "rtwtypes.h"];
61 ignore_files_h = ["Product_parameter.h","rtmodel.h", "rtwtypes.h"];
62 ignore_files_c = "Product_parameter.c";
63 % get all auto generated .h and .c files
64 files_h = dir([build_dir '*.h']);
65 files_c = dir([build_dir '*.c']);
66 % get all important files and replace Embedded Coder header file by ZUSE
      header file with same content
67 for i = length(files_h):-1:1
68     if sum(ismember(ignore_files_h, files_h(i).name)) > 0
69         files_h(i) = [];
70     else
71         find_replace([build_dir files_h(i).name], '#include
      "rtwtypes.h"', '#include <stdint.h>', '#include <stdbool.h>');
72         copyfile('temp.c', [bdroot '\ ' files_h(i).name]);
73         delete 'temp.c'
```

```
74     end
75 end
76 for i = length(files_c):-1:1
77     if sum(ismember(ignore_files_c, files_c(i).name)) > 0
78         files_c(i) = [];
79     else
80         find_replace([build_dir files_c(i).name], '#include
            "rtwtypes.h"', '#include <stdint.h>', '#include <stdbool.h>');
81         copyfile('temp.c', [bdroot '\ ' files_c(i).name]);
82         delete 'temp.c'
83     end
84 end
85
86 %% CMakeList
87 fprintf('\n\n');
88 fprintf('-----\n');
89 fprintf(' for CMakeList in alphabetical order \n');
90 fprintf('-----\n');
91 ls = {};
92 for i = 1:1:length(files_c)
93     ls{end+1} = files_c(i).name;
94 end
95
96 for i = 1:1:length(files_h)
97     ls{end+1} = files_h(i).name;
98 end
99 ls{end+1} = 'Playground.c';
100 ls{end+1} = 'Playground.h';
101 ls = sort(ls);
102 for i = 1:1:length(ls)
103     disp(ls(i))
104     %disp('/n')
105 end
106
107 %% Finished
108 fprintf('\n\n');
109 fprintf('-----\n');
110 fprintf(' Code-Generation finished \n');
111 fprintf('-----\n');
112
```

```
113 %% Functions
114 function find_replace(file, old_h, int_h, bool_h)
115     fidi = fopen(file,'r');
116     fido = fopen('temp.c','w');
117     try
118         while ~feof(fidi)
119             l = fgetl(fidi); % read line
120             if contains(l,old_h)
121                 l = replace(l, old_h, int_h); % modify header line
122                 fprintf(fido,'%s\n',l); % write new line with int header
123                 fprintf(fido,'%s\n',bool_h); % write extra bool header
124             else
125                 l = strrep(l,'float32_t','float'); % if present replace old
                    datatype with new (until Matlab 2023)
126                 l = strrep(l,'float64_t','double'); % if present replace old
                    datatype with new (until Matlab 2023)
127                 l = strrep(l,'bool_t','bool'); % if present replace old datatype
                    with new (until Matlab 2023)
128                 fprintf(fido,'%s\n',l); % write new line
129             end
130         end
131     catch
132         fclose(fido);
133     end
134     fclose(fidi);
135     fclose(fido);
136 end
```

Skript A.1: Skript für die Autocodegenerierung und Nachbearbeitung

## B Abkürzungen Matlab-Modell

| Abkürzung | Bezeichnung                  | Zusatzinfo              | neu? |
|-----------|------------------------------|-------------------------|------|
| Akku      | Akkumulator                  |                         | Ja   |
| Aktv      | aktivieren, aktiv            |                         | Nein |
| Anm       | Anlaufmodus                  |                         | Nein |
| Aus       | aus, ausschalten             |                         | Nein |
| Blk       | blockieren, Blockade         |                         | Ja   |
| Btr       | Betrieb                      |                         | Nein |
| bus       | Bus-Signal                   | Spezieller Variablentyp | Nein |
| Dek       | dekrementieren, Dekrement    |                         | Nein |
| dm        | Drehmoment                   | physikalische Größe     | Nein |
| Ein       | ein, einschalten             |                         | Nein |
| Elk       | Elektrik, Elektronik         |                         | Ja   |
| Erk       | Erkennung                    |                         | Nein |
| flg       | Flag                         | Spezieller Variablentyp | Nein |
| Ink       | inkrementieren, Inkrement    |                         | Nein |
| Lim       | Limit                        |                         | Ja   |
| lst       | Leistung                     | physikalische Größe     | Ja   |
| Max       | Maximal                      |                         | Nein |
| Mot       | Motor, Maschine              |                         | Nein |
| Mtst      | Motorsteuerung               |                         | Nein |
| n         | Drehzahl                     | physikalische Größe     | Nein |
| Nsr       | not Safety relevant          |                         | Ja   |
| num       | Nummer, Zahl, Ziffer, Anzahl | Spezieller Variablentyp | Nein |
| Ok        | In Ordnung, Ok               |                         | Ja   |
| Red       | reduzieren, Reduzierung      |                         | Ja   |
| spng      | Spannung                     | physikalische Größe     | Nein |
| Sr        | Safety relevant              |                         | Ja   |
| stm       | Strom                        | physikalische Größe     | Nein |
| Stst      | Stillstand                   |                         | Ja   |
| t         | Zeit                         | physikalische Größe     | Nein |
| tmp       | Temperatur                   | physikalische Größe     | Nein |
| Zael      | Zähler                       |                         | Nein |
| Zn        | zu niedrig                   |                         | Ja   |

Tab. B.1: Abkürzungsverzeichnis Matlab - Modell